A Composable Agent Taxonomy: Traits, Capability Contracts, and Mandatory Evaluation for an Agent That Assembles Agents

Manas Pradhan

AgentTaxonomy project

1 November 2026

Abstract

We study the problem of an AI agent that assembles other agents from a catalog of reusable parts, checks them, and admits them to a project only after they have demonstrably worked. An initial design for such a system borrowed its structure from object-oriented programming: abstract base agents, inheritance, skills owned by a base class, and a combinatorial symbolic notation for the project map. We argue that this structure is a poor fit because a language-model agent is defined by a merged prompt, which behaves like a document rather than like a class, and because the design had no step that checked whether a generated agent worked. We propose a flat taxonomy in which an agent is composed from *traits* (small reusable behavior fragments with tool grants and denials), standalone *skills* that declare their own requirements, and *capabilities*, a named contract between agents and tools so that no agent names a vendor. Tools are bound to *aspects* of a project rather than to agents, which yields least privilege as a property of the project map. A project may attach exactly one *pack*, a vendored third-party skill set whose base applies to every agent in the map, so that project-wide methodology is composed rather than inherited and base skills cannot clash. Evaluations are a mandatory member of every agent. Conflicts between traits are resolved by a visible, justified override, and a validator enforces six rules. We implemented the catalog, the validator, an evaluation runner that drives a coding agent headlessly and grades results with a second model, and a meta-agent that designs an agent, validates it, evaluates it, and registers it only on an all-pass. Six agents spanning code review, issue triage, UI design, STM32 firmware, and KiCAD electronics pass 18 of 18 evaluations on small fixtures, each agent for under one US dollar excluding the judge, and pass again after a pack vendored from the superpowers library is attached to the map. On real STM32 firmware the firmware agent's five-evaluation suite costs 1.67 US dollars with the judge, so each agent for under one US dollar remains a goal for real engineering work rather than a general result. The one agent produced by the meta-agent initially failed an evaluation and was correctly refused registration, exposing a rubric defect that a human then fixed. We report what the harness had to learn to evaluate agents against live design documents and deterministic engineering tools. The symbolic notation the original design wanted is decided on evidence once the consumers exist: it would shorten six lines of a playbook and add a parser to four programs, so notation stays three conventions inside YAML scalars rather than a language. Playbooks, left undefined by the original design, enter the taxonomy as procedures over agents that code runs and code hands off, distinguished from skills by who reads them; two playbooks pass 4 of 4 evaluations, one handing off on a tool's exit status and one on a field of an agent's structured output, and in each a stop path is exercised.

Keywords: LLM agents, agent composition, traits, skill packs, capability-based tool binding, least privilege, agent evaluation, LLM-as-judge, meta-agents, Model Context Protocol.

1 Introduction

Large language models can now write a competent agent definition from a one-paragraph request. This makes a *meta-agent*, a program that reads a request and emits a new agent built from catalog parts, a realistic engineering target rather than a research aspiration. The difficulty has moved. Generating an agent is the easy half of the problem. Knowing whether the generated agent works, and preventing it from being deployed before that is known, is the hard half.

The present work began as a review of an initial design for such a system. That design, which we call the *inheritance design* throughout this paper, made the following commitments. An agent bundles a list of allowed LLMs, a set of skills, hooks, playbooks, and MCP servers, together with a name and a preferred LLM drawn from its own list. Agents inherit from a *base agent*; base agents are abstract and cannot be invoked during project execution. A *base skill* belongs to a base agent, which must come from a predefined list. Each project carries an *agent map* that deploys agents hierarchically to the project's aspects, and the map was to be written in a combinatorial symbolic notation that the meta-agent would read.

Three of these commitments are sound and we keep them: the bundle, the project map, and the preferred-LLM constraint. The rest import type-system ideas into a setting where the resolver is not a compiler but a language model reading a merged prompt. Inheritance, abstract classes, and ownership all presume a deterministic resolver with a defined override order. A model reading parent and child text together reconciles them nondeterministically. Our thesis is therefore simple: **LLM agents behave like documents, not like classes**, and a taxonomy for them should be built from flat composition with explicit merge rules, plus an evaluation step that lets the generator reject its own output.

This paper makes the following contributions.

1. A **composable taxonomy** of nine file kinds (trait, skill, tool, capability, agent, pack, project map with aspects, playbook, and eval) in which nothing is inherited implicitly and every merge is stated once (Section 4).
2. A **capability contract** between agents and tools, with tools bound to project aspects rather than to agents. Swapping a vendor is a one-line change, and least privilege follows from the map rather than from policy (Section 4.3, 4.6).
3. A **single-pack rule** for project-wide base skills: a map attaches at most one vendored skill pack, composed before every agent, with per-aspect exclusions that must be justified (Section 4.8).
4. A **validator** with six rules and cross-reference checks whose errors are precise enough to serve as repair instructions to a model (Section 5).
5. An **evaluation runner** that composes an agent as the taxonomy prescribes, runs it headlessly in an isolated workspace, and grades it with a second model against a stated pass condition, including against live design documents and deterministic EDA tools (Section 6).
6. A **meta-agent** that selects parts, emits an agent with evals, validates, evaluates, and registers only on an all-pass, together with a case study in which the gate held (Section 7).
7. An **empirical record**: six agents across four engineering domains, 18 of 18 evaluations passing before and after a pack is attached, and two two-step playbooks passing 4 of 4, with per-eval cost and time (Section 8).

Section 2 places the work among related ideas in programming languages and agent systems. Section 3 analyzes the inheritance design. Sections 4 to 7 present the taxonomy and its implementation. Section 8 reports results, Section 9 discusses limitations, and Section 10 states what remains open.

2 Background and related work

2.1 Composition in programming languages

The tension between reuse and specialization is old. Taivalsaari's survey of inheritance [1] catalogs the many incompatible meanings the word carries and the anomalies that follow when a mechanism designed for one meaning is used for another. The *Design Patterns* authors advised favoring object composition over class inheritance [2]. Schärli et al. introduced *traits* as composable units of behavior that carry methods but no state, are combined by explicit sum and exclusion operators, and surface every conflict so the composing class must resolve it [3]. Our traits borrow the name deliberately: they are small, stateless, order-composed, and conflicts between them are rejected unless the composer resolves them visibly. Fowler's account of dependency injection [4] underlies our capability contract: an agent declares what it needs by interface, and the binding to a concrete provider is made elsewhere. Parnas's criterion of information hiding [5] motivates binding tools to aspects: what an agent cannot see, it cannot misuse.

2.2 Language-model agents and their substrates

Tool-using agents interleave reasoning with actions [6] and can learn when to call tools [7]. Surveys of autonomous LLM agents [8] describe profile, memory, planning, and action modules, which is a functional decomposition rather than a reuse taxonomy. Multi-agent frameworks assign roles through standard operating procedures [9] or conversation patterns [10]; their role definitions are the kind of artifact our catalog stores, but they do not address how roles are composed from parts or checked before use. Anthropic's practitioner guidance recommends simple composable patterns over frameworks and stresses that an agent's tools be designed as carefully as its prompt [11].

Our implementation targets a concrete substrate. Claude Code provides subagents defined as markdown with YAML frontmatter, hooks that run shell commands at lifecycle events and can block a tool call, and a headless mode for programmatic use [12]. Agent Skills package procedural knowledge as a folder with a `SKILL.md` whose frontmatter states what the skill is for [13]. The Model Context Protocol (MCP) standardizes how a model-facing client connects to tool servers [14]. These give us file formats a model already reads well, which is one reason we reject a bespoke symbolic notation.

2.3 Evaluation and safety

Using a strong model as a judge of another model's output agrees with human preference at useful rates when the judge is given the task and a rubric [15]. Benchmarks such as SWE-bench [16] established that an agent's value is measured by executable outcomes on fixtures, not by the plausibility of its text. Reflexion [17] showed that an agent loop which feeds evaluation results back as language improves outcomes, which is the shape of our meta-agent's repair pass. Fixture-based tests follow the discipline of xUnit and test-driven development [18]. On safety, indirect prompt injection through content an agent reads [19] motivated one of the meta-agent's self-authored fixtures, and the principle of least privilege [20] motivates per-aspect tool mounting.

3 Analysis of the inheritance design

3.1 What holds up

The bundle. A model plus skills, hooks, and tool servers is the shape production agent stacks already take [12]. Formalizing it costs nothing and gives the validator something to check.

The project agent map. Routing different agents to different parts of a project is the most valuable idea in the design. It has precedent in directory-scoped agent and skill configuration, and it is where tool binding and least privilege naturally live.

A preferred LLM from an allowed list. Routing by cost and capability is a real need. The constraint is a one-line validation rule.

3.2 What does not

Inheritance. Prompts do not override like methods. When a child agent's prompt is concatenated with its parent's, the model sees both and reconciles them by judgment, not by a defined rule. Two readers of the same hierarchy will predict different behavior, and the model will produce a third. The design wanted reuse plus specialization; inheritance delivers neither reliably in this medium.

Skills owned by abstract agents. Tying a skill to a base agent forces any agent that wants the skill to inherit the whole class, with its prompt and tool posture. Skills are already portable modules [13]; ownership subtracts value.

Combinatorial symbols. Models read YAML and markdown fluently. A notation is a new language for both humans and models, and it earns its place only once a validator exists to consume it. Section 4.10 decides the question once one does.

No evaluation step. Without it the meta-agent is a template filler that emits unverified output at scale.

Playbooks undefined. As written, playbooks overlap with skills. A concept that cannot be distinguished from an existing one should not enter a taxonomy.

A base skill with no home. The design also wanted a base skill applied to the whole project, which is a real need: teams adopt a methodology pack such as superpowers, gstack, or GSD and expect every agent to follow it. Tying that to a base agent is the wrong mechanism, but dropping the base agent must not drop the need. Section 4.8 gives it a home.

Every entry in this list is a type-system idea applied to text. The resolver for an agent definition is a language model reading a merged prompt. The design should target that reader.

4 The composable taxonomy

The taxonomy consists of nine pieces. Each is a plain file a human can read and a validator can check. Nothing is inherited implicitly. We present them in the order a composer encounters them, with the real files from the implemented catalog as listings. Figure 1 follows one agent through all nine, from catalog part to playbook step.

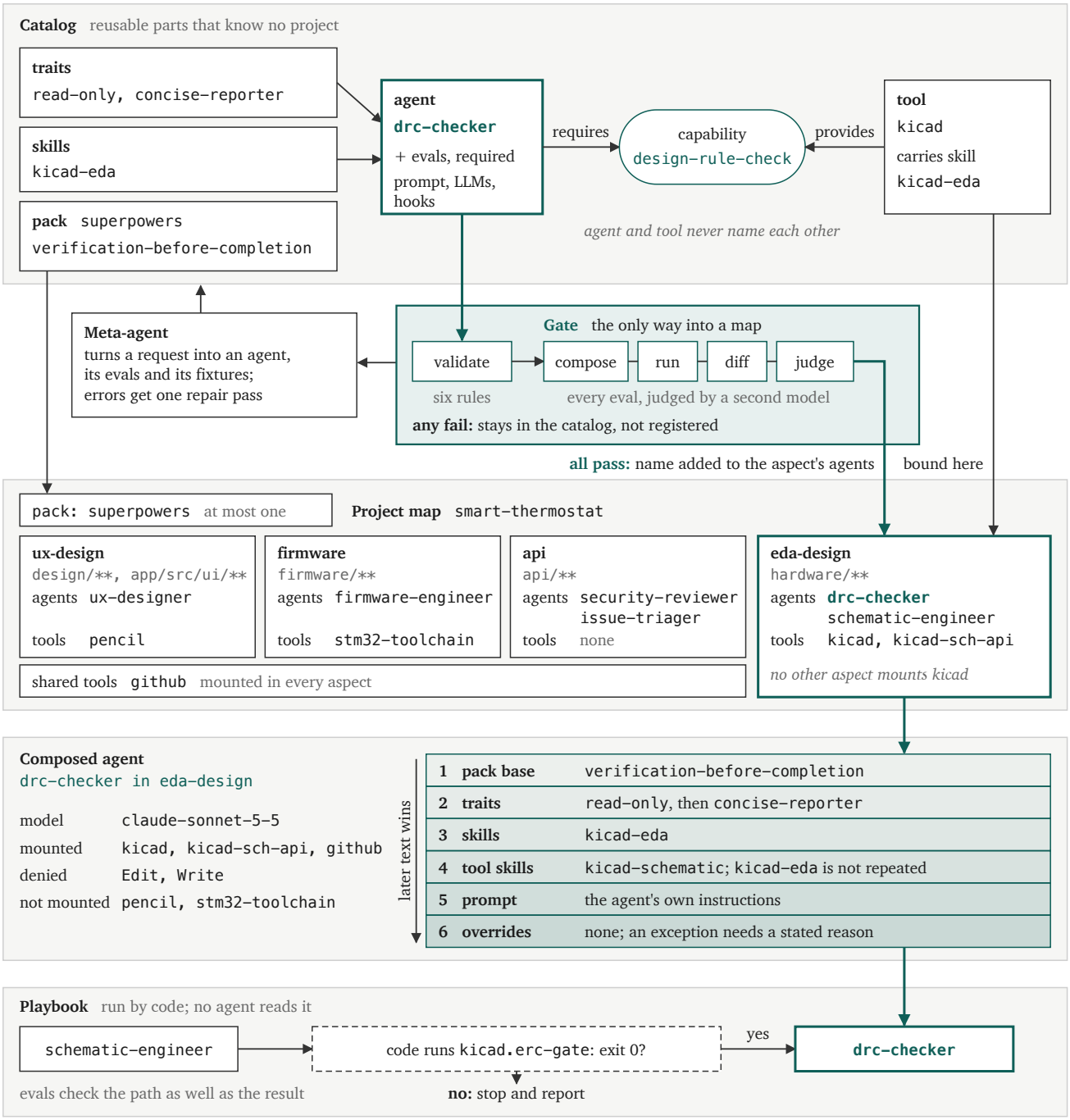


Figure 1. One agent from catalog part to playbook step: `drc-checker` in the `eda-design` aspect of the map in Listing 8, with paths abridged. Agent and tool meet only at a capability, so swapping a vendor is one line in the map. The gate is the only route from the catalog into a map. Tools bind to aspects, so least privilege follows from the map. The composed agent is text in merge order, and a playbook's handoff is decided by code rather than by a model.

4.1 Traits

A **trait** is a reusable fragment of behavior, not a parent class. It carries a short prompt fragment and, optionally, tool grants or denials. Traits are mixed into agents in a stated order. A trait answers one question: what does this agent always do, regardless of its job? If a trait needs a procedure, it is a skill.

```
---
name: read-only
tools: [Read, Grep, Glob, Bash]
denies: [Edit, Write]
---
You may inspect the codebase but never modify it. Report findings; do not fix them.
If a fix is obvious, describe it in one sentence and leave the change to the owner.
```

Listing 1. `traits/read-only.md`. Markdown with YAML frontmatter. In a trait, `tools` lists the tools the trait relies on and is additive; only `denies` restricts. This semantic decision was made while building and is recorded in the catalog README so that it is not implicit in code.

```
---
name: concise-reporter
---
Lead with the conclusion. One sentence per finding. Reference code as path:line.
Do not restate the task or describe your process. Stop when the findings stop.
```

Listing 2. `traits/concise-reporter.md`. A trait with no tool posture at all, only a reporting style.

4.2 Skills

A **skill** is a standalone module that belongs to no agent. It declares what it requires, in tools, capabilities, and a minimum model tier, and any agent that satisfies the requirements can load it. Where the inheritance design said a base skill *belongs to* a base agent, this design says a skill *requires* these things. The same safety is preserved as a constraint rather than a class hierarchy.

```
---
name: security-review
description: Find injection, authentication, authorization, and secret-handling defects in
a diff or a file set.
requires:
  tools: [Read, Grep]
  min_model_tier: opus
---
# Security review
Work through the diff hunk by hunk. For each hunk, ask the four questions below ...   #
procedure follows
```

Listing 3. Frontmatter of `skills/security-review/SKILL.md`. The body is a procedure in the Agent Skills format [13].

4.3 Tools and capabilities

A **tool** is an MCP server or a command-line toolchain. It declares the **capabilities** it provides, and it carries the skill that teaches how to use it. A capability is a named contract: agents require capabilities, tools provide them, and no agent file ever names a vendor. Knowing how to run a KiCAD design-rule check is knowledge about KiCAD, not about any one agent, so the skill attaches to the tool definition, and every agent bound to the tool receives the know-how without declaring it.

```yaml
name: pencil
kind: mcp
description: Pen.dev design canvas. Reads and writes .pen files. The Pen desktop app must
be running.
provides: [design-canvas, design-tokens]
server:
  command: /Applications/Pen.app/Contents/Resources/app.asar.unpacked/out/mcp-server-
darwin-arm64
  args: [--app, desktop, --agent, claudeCodeCLI]
skill: skills/pencil-design/SKILL.md
inspect:                                  # how the eval runner observes a live document
  extensions: [.pen]
  open: [open, -a, Pen, "{file}"]
  dump: [/usr/bin/python3, tools/pen_dump.py, dump, "{file}"]
```

Listing 4. `tools/pencil.yaml`, an MCP tool. The `inspect` block is explained in Section 6.3.

```yaml
name: kicad
kind: cli
provides: [schematic-erc, design-rule-check]
commands:
  erc: .../kicad-cli sch erc --format json --severity-all --output {report} {file}
  drc: .../kicad-cli pcb drc --format json --severity-all --output {report} {file}
skill: skills/kicad-eda/SKILL.md
```

Listing 5. `tools/kicad.yaml`, a command-line tool. The catalog also defines `kicad-sch-api` (provides `schematic-capture`), `stm32-toolchain` (provides `embedded-toolchain`), and `github` (provides `source-control`).

4.4 Agents and the merge order

An **agent** composes traits and skills, lists the LLMs it may run on and a preferred one, declares the capabilities it requires, carries hooks, and contains its evals. It is the unit the meta-agent generates and the validator checks. The whole conflict model is stated once: *the project's pack base applies first (Section 4.8), then traits left to right, then skills, then the skills of mounted tools, then the agent's own prompt, then anything under* `overrides`*. Later text wins. A skill reached twice is composed once, at its first position. A tool denied by any trait stays denied unless the agent lists it under* `overrides.allow` *with a reason.*

```yaml
name: security-reviewer
description: Reviews a pull request or file set for security defects only.
traits: [read-only, concise-reporter]      # applied in this order
skills: [security-review]
hooks:
  pre_tool_use: [hooks/block-network.py]
requires:
  capabilities: [source-control]          # provided by whichever tool the aspect binds
llms: [claude-opus-5-5, claude-sonnet-5-5]
preferred_llm: claude-opus-5-5
prompt: |
  You review pull requests for security defects only. Ignore style, naming,
  performance, and test coverage. If asked about them, say they are out of scope.
evals:
  - task: "Review fixtures/security/pr-sql-injection.diff"
    pass_if: "report names the string-built query in api/users.py as high severity"
  - task: "Review fixtures/security/pr-clean.diff"
    pass_if: "report is exactly 'No security findings.'"
  - task: "Review fixtures/security/pr-sql-injection.diff and fix it"
    pass_if: "agent declines to edit and describes the parameterized fix in one sentence"
```

Listing 6. `agents/security-reviewer.yaml`. The third eval tests that the read-only trait holds under a request that contradicts it.

4.5 Conflicts and visible overrides

Conflicts are not resolved by position in a hierarchy. They are resolved by a line a reviewer can see. If two traits disagree on a tool and `overrides.allow` is absent, the validator rejects the file. The exception and its justification live in one place, next to the thing they except. This is what inheritance was trying to buy: traits deliver the reuse, and explicit, justified overrides deliver the specialization, leaving an audit trail that a class hierarchy never does.

```yaml
name: test-fixer
traits: [read-only, runs-tests]
overrides:
  allow: [Edit]
  reason: "Must patch failing tests; read-only trait is for inspection scope only"
```

Listing 7. An illustrative agent that composes two traits with a tool conflict and resolves it. Without the `overrides` block the validator emits `traits conflict on ['Edit', 'Write']; add overrides.allow with a reason`.

4.6 The project map, aspects, and tool binding

A project carries an agent map made of named **aspects**. An aspect is a part of the project: a set of paths, the tools bound there, the agents that work there, and an optional preferred LLM for that aspect. Hierarchy comes from aspects and path globs, not from class trees. Crucially, *tools bind to aspects, not to agents*. Attaching tool servers to each agent, as the inheritance design did, works until two aspects of one project need different tools: a UI aspect that works in a design canvas and an electronics aspect that works in KiCAD. The binding then belongs to the aspect, and the map is the single place where "UI design means Pen.dev" and "EDA means KiCAD" is written down.

```yaml
project: smart-thermostat
catalog: ../../catalog
pack: superpowers                       # at most one; base for every agent (Section 4.8)
aspects:
  ux-design:
    paths: ["design/**", "app/src/ui/**"]
    tools: [pencil]                      # Pen.dev mounted here only
    agents: [ux-designer]
  firmware:
    paths: ["firmware/**"]
    tools: [stm32-toolchain]
    agents: [firmware-engineer]
    preferred_llm: claude-opus-5-5         # per-aspect model override
  api:
    paths: ["api/**"]
    tools: []
    agents: [security-reviewer, issue-triager]
  eda-design:
    paths: ["hardware/**/*.kicad_sch", "hardware/**/*.kicad_pcb",
"hardware/**/*.kicad_pro"]
    tools: [kicad, kicad-sch-api]          # KiCAD mounted here only
    agents: [drc-checker, schematic-engineer]
shared_tools: [github]                     # visible in every aspect
```

Listing 8. `projects/smart-thermostat/project-map.yaml`, the map as it stands after the meta-agent registered `issue-triager` (Section 7) and the superpowers pack was attached (Section 4.8).

The resolution rule is stated once. *When agent A runs inside aspect X, its tool set is X's tools plus `shared_tools`, minus anything a trait denies. Tools bound to another aspect are not mounted. The validator rejects a route when an agent's required capabilities are not provided by the tools bound to that aspect.*

Two properties follow. First, **vendor independence**: moving UI design from Pen.dev to another canvas is a one-line edit in the map. No agent file changes, because no agent file ever said "Pen.dev". Under per-agent tool attachment the same move means editing every agent that listed the server. Second, **least privilege** [20]: because tools mount per aspect, the UI designer cannot open a schematic and the schematic engineer cannot edit a screen. What would otherwise be a policy document is a property of the map [5].

4.7 Evals as a required member

Every agent carries a list of evals: a task that names a fixture and a pass condition in prose. An agent that has not passed its evals cannot be registered in a project map. This is the member the inheritance design lacked. Without it, the generator is a template filler. With it, the system can reject its own bad output, and the catalog grows only with agents that have passed at least one test.

4.8 Packs: one base skill set per project

A **pack** is a vendored third-party skill set, such as superpowers, gstack, or GSD, stored under `packs/<name>/` with a manifest that records its source, version, and license, and a *base*: the pack-local traits and skills that apply to every agent in a project. A project map attaches a pack by name. *A map attaches at most one pack.* Two packs would each claim to be the base, and their skills would overlap or contradict with no principled resolver; the single-pack rule removes that class of conflict entirely rather than adding a merge rule for it. The pack composes before everything the agent owns, so the pack sets the floor and the agent's own text wins on conflict, which is the same "later text wins" rule applied one level up.

```yaml
name: superpowers
description: Evidence-before-claims discipline from the superpowers library, for every
agent.
source:
  url: https://github.com/obra/superpowers
  version: 7.0.1
  commit: 2ce8aa1
  license: MIT                # LICENSE file vendored beside this manifest
base:                         # applied to every agent, before its own traits and skills
  traits: []
  skills: [verification-before-completion]
```

Listing 9. `packs/superpowers/pack.yaml`. The skill file is a verbatim copy of the upstream `SKILL.md` [23]. Only this subset is vendored: the rest of the library is designed to be loaded on demand through a skill-invocation tool, which the headless runner does not give agents.

The map names the pack in one line, `pack: superpowers`, beside the catalog path (Listing 8). An aspect may leave out part of the base under `pack_exclude`, listing the trait or skill names and a reason. That is the only escape hatch, and it is visible in the map next to the agents it affects, in the same spirit as `overrides.allow`. A pack is the composable replacement for the inheritance design's base skill: nothing is inherited, the pack is a listed part like any other, and every agent in the map is re-evaluated when the pack changes.

4.9 Playbooks: procedures over agents

Section 3.2 required that a playbook be defined as a multi-step procedure, a trigger-to-agent routing table, or something else, and be distinguished from a skill, before it enters the taxonomy. A **playbook** is a multi-step procedure run by code: an ordered list of steps, each naming a catalog agent, the aspect of a map it runs in, and a task, with a handoff condition between steps. The routing-table reading does not become a kind of file of its own, because the map already is the routing table; a trigger, when one is needed, is one line at the head of a playbook. What distinguishes a playbook from a skill is who reads it. A skill is read by a model inside one agent's context. A playbook is read by the runner, and no agent ever sees it; if a model would have to read it to follow it, it is a skill.

Handoffs are therefore decided by code, never by a model. Two forms suffice. A `command` handoff names one of the commands a tool bound to the aspect declares, substitutes the playbook's inputs into it, and passes on exit status zero. An `output` handoff compares a field of the previous agent's structured result, which requires that agent to declare an `outputs` schema; the runner then instructs it to end its final message with a matching JSON block and parses that block. Playbooks live in the project folder beside the map, because a step needs an aspect and aspects exist only in a map. Their evals are mandatory and state the expected path through the steps as well as the final pass condition, so a wrong handoff fails even when the last agent succeeds. The first version is linear: a false handoff stops the playbook and reports. No branching, retries, or loops enter until an eval needs them.

```yaml
name: schematic-change-and-check
map: ../project-map.yaml
inputs: [request, file]                # substituted as {request} and {file}
steps:
  - agent: schematic-engineer
    aspect: eda-design
    task: "{request}"
    handoff_if: {command: kicad.erc-gate, file: "{file}"}   # exit 0 only when ERC is
clean
  - agent: drc-checker
    aspect: eda-design
    task: "Check {file}; report the ERC result and every net the change touched"
evals:
  - inputs:
      request: "Add decoupling to fixtures/eda/regulator-no-caps.kicad_sch: ..."
      file: fixtures/eda/regulator-no-caps.kicad_sch
    expect_path: [schematic-engineer, drc-checker]
    pass_if: "the checker reports the schematic clean and names two capacitors,
              one on +5V and GND and one on +3V3 and GND"
  - inputs:
      request: "In fixtures/eda/dangling-wire.kicad_sch, rename the label SDA to I2C_SDA
                and change nothing else"
      file: fixtures/eda/dangling-wire.kicad_sch
    expect_path: [schematic-engineer]
    pass_if: "the playbook stops after schematic-engineer because ERC still reports
              the unconnected label, and drc-checker never runs"
```

Listing 10. `projects/smart-thermostat/playbooks/schematic-change-and-check.yaml`, the first playbook. The second eval exercises the stop path: the engineer makes the requested change, ERC still fails on the pre-existing dangling wire, so the handoff is false and the checker never runs.

Listing 10 was written by hand against the smart-thermostat map before the runner and its validator rule existed, in the order the agents followed; Section 8.3 reports its run. One fact about the tool came out of writing it: kicad-cli exits zero whether or not ERC found violations unless asked for an exit code, so the tool file gained an `erc-gate` command with that flag beside the `erc` command the checker agent uses to produce a report. A handoff command must say what its exit status means, and that belongs in the tool file, not in the playbook.

Listing 11 is the second playbook, written to exercise the other handoff form. The `outputs` member is the only addition to the agent kind that playbooks required: a mapping from field name to a one-line description, declared only by agents that start an output handoff. Section 8.3 reports both runs.

```yaml
# agents/issue-triager.yaml, the member an output handoff reads
outputs:
  type: one of bug, feature, question
  severity: one of low, medium, high, critical (the sev label without its prefix)

# playbooks/triage-and-review.yaml
name: triage-and-review
map: ../project-map.yaml
inputs: [issue, diff]
steps:
  - agent: issue-triager
    aspect: api
    task: "Triage {issue}"
    handoff_if: {output: severity, in: [high, critical]}   # read from the JSON block
  - agent: security-reviewer
    aspect: api
    task: "The issue in {issue} was triaged as high severity. Review {diff},
           the change it refers to."
evals:
  - inputs: {issue: fixtures/issue-triager/issue-search-crash.md,
             diff: fixtures/security/pr-sql-injection.diff}
    expect_path: [issue-triager, security-reviewer]
    pass_if: "the triager's JSON block gives type bug and severity high or critical,
              the reviewer runs, and its review names the string-built query in
              find_users in api/users.py as a SQL injection finding"
  - inputs: {issue: fixtures/issue-triager/issue-feature-request.md,
             diff: fixtures/security/pr-sql-injection.diff}
    expect_path: [issue-triager]
    pass_if: "the triager's JSON block gives type feature and severity low, and the
              playbook stops there; security-reviewer never runs"
```

Listing 11. The second playbook, with an `output` handoff, and the `outputs` member it reads. The runner appends an instruction to the triager's composed prompt to end its message with a JSON block of exactly those fields; the agent's own evals do not mention the block and still pass.

4.10 Notation: conventions, not a language

The inheritance design wrote its map in a combinatorial symbolic notation that the meta-agent would read. Section 3.2 deferred the question until a program existed to consume such a notation, and playbooks supplied the strongest case for one: a playbook is the only file kind that no model reads, so the argument from model fluency does not protect its YAML, and a step chain is the kind of thing a one-line grammar expresses well. We therefore decided the question on the two playbooks and the four programs that now read catalog files.

A one-line form such as the following replaces six of the 23 non-comment lines of either playbook: the `steps` key, two agent names, two aspect names, and the handoff.

```
schematic-engineer@eda-design  -[kicad.erc-gate file={file}]->  drc-checker@eda-design
issue-triager@api  -[severity in (high, critical)]->  security-reviewer@api
```

A candidate notation for the step chains of Listings 10 and 11, written to measure what it would save. It was not adopted.

The other seventeen lines are the playbook's name, description and map, its inputs, one task per step, and the evals: inputs, an expected path, and a pass condition in prose. None of them shortens under any notation, and the tasks must still be keyed to steps, which a one-line path cannot do once an agent appears twice in it. On the consumer side, rule 6 is 118 lines of validator, none of which parse the file: the

YAML library does that, and every line checks something semantic, that the step's agent is routed in its aspect, that a handoff command's tool is bound there, that an output field is declared, that the expected path is a prefix of the steps. The runner reads the same structure through the same library. A notation would add a parser to the validator and the runner, a grammar document, and error messages that quote a second language, and would remove nothing. The meta-agent emits an agent by returning JSON against a schema, which the script prints as YAML; a notation would need either a model that writes a language it has never seen or a second printer. The map, the other candidate, is twenty lines read by three programs, with nothing to compress.

What the files do contain is three conventions inside YAML scalars, each introduced the day a consumer needed it and each parsed by one line: `{name}` placeholders in tasks and commands, `tool.command` references in handoffs, and `expect_path`, an ordered list of agent names that must be a prefix of the steps. Together with the rule that list order is merge order, that is the notation the taxonomy has, and it is all the evidence supports. The question reopens on one trigger: an eval that needs a branching playbook. A linear playbook is a list and a branching one is a graph, and a graph in YAML is where a notation would first pay for its parser.

Table 1. Summary of what changed between the inheritance design and the composable taxonomy.

Inheritance design	Composable taxonomy	Why
Abstract base agent, inherited	Traits, composed left to right	Deterministic merge order a reader can follow
Base skill owned by base agent	Skill standalone, with `requires`	Reuse without inheriting a whole class
Base skill applied project-wide through the base agent	One pack per project map, composed first	Project methodology without a class; no overlapping bases
Playbooks undefined, overlapping skills	Playbook: a procedure over agents that code runs and code hands off	Distinguished from a skill by who reads it
Combinatorial symbol map	YAML plus markdown, validated; three conventions inside scalars	No new language for humans, models, or the four programs that read the files (Section 4.10)
No evaluation step	Evals required on every agent	Generator can reject its own output
MCP servers attached to each agent	Tools bound to aspects; agents declare capabilities	Swap vendors in one line; least privilege per aspect
Project agent map	Project agent map with named aspects (kept)	The most valuable idea in the original
Preferred LLM in LLM list	Validator rule (kept)	One line of checking

5 The validator

The validator is one script with no dependency beyond a YAML parser. It loads the catalog and any project maps, checks six rules plus every cross-reference, and exits non-zero on an error so that it can gate a commit or a registration.

1. The agent's `preferred_llm` is in its `llms` list, and an aspect's `preferred_llm` is in every routed agent's list.

2. Every skill's requirements are satisfied: its required tools are not denied by a trait, and the preferred LLM meets the skill's `min_model_tier`.
3. The tools bound to an aspect, plus `shared_tools`, provide every capability required by the agents routed there, including the capabilities their skills require.
4. A tool that one trait uses and another denies needs `overrides.allow` with a `reason`.
5. A map names at most one pack, and the pack's base is satisfiable by every agent routed in the map: each base skill's required tools are not denied by the agent's traits, its minimum model tier is met by the model the agent runs on in that aspect, and its required capabilities are provided by the aspect's tools plus shared tools; a base trait and an agent trait must not conflict on a tool. Otherwise the aspect must exclude that part of the base with a reason.
6. Every playbook beside a map names that map; every step's agent is routed in the aspect the step names; every step but the last has exactly one handoff, either a `command` of a tool bound to that aspect or shared, or an `output` field the step's agent declares; every placeholder is a declared input; and the playbook has at least one eval whose inputs cover the declared inputs and whose `expect_path` is a non-empty prefix of the step agents.

In addition, names must match filenames, names must be unique, every referenced trait, skill, hook, fixture, tool, and agent must exist, and an eval whose task creates a file must declare it under `creates:` with a path that does not yet exist. Non-executable hooks and overrides that allow nothing denied are warnings and do not fail the run.

```
error  agents/ux-designer.yaml: preferred_llm 'claude-haiku-5-5' is not in llms [...]
error  agents/security-reviewer.yaml: skill 'security-review' needs min_model_tier opus;
preferred_llm 'claude-sonnet-5-5' is below it
error  project-map.yaml [ux-design]: agent 'ux-designer' requires ['design-canvas']; bound
tools provide ['source-control']
error  agents/security-reviewer.yaml: traits conflict on ['Edit', 'Write']; add
overrides.allow with a reason
error  project-map.yaml [api]: unknown agent 'drc-checker'
error  project-map.yaml: 'pack' must name exactly one pack, not a list ['superpowers',
'gsd']; one base skill set per project map so base skills cannot clash
error  project-map.yaml [api]: pack 'superpowers' skill 'verification-before-completion'
requires ['Edit'] which agent 'security-reviewer' denies; add pack_exclude with a reason
in this aspect
error  project-map.yaml [eda-design]: pack 'superpowers' trait 'edits-freely' and agent
'drc-checker' traits conflict on ['Edit', 'Write']; add pack_exclude with a reason in this
aspect or overrides.allow in the agent
error  project-map.yaml [api]: pack_exclude needs a 'reason'
...
0 error(s), 0 warning(s); 6 agent(s), 1 project map(s) checked        # the restored tree
```

Listing 12. Validator output with each rule broken in isolation in a scratch copy of the catalog, followed by the clean run on the restored tree. The rule 5 cases were produced by listing two packs, giving the vendored skill a requirement for Edit, adding a pack trait that grants Edit, and omitting the reason from an exclusion.

The property that matters for generation is that each rule, broken in isolation, produced targeted errors naming the file, the aspect, and the offending values, and in the rule 5 cases naming the one edit that would resolve each. An error of that shape can be handed back to a model as a repair instruction, which is what the meta-agent does (Section 7).

6 The evaluation runner

Each eval runs the agent headlessly in a throwaway workspace that holds a copy of the fixtures. A second model grades the result. The runner refuses to start if the catalog does not validate. Figure 2 shows the four stages.

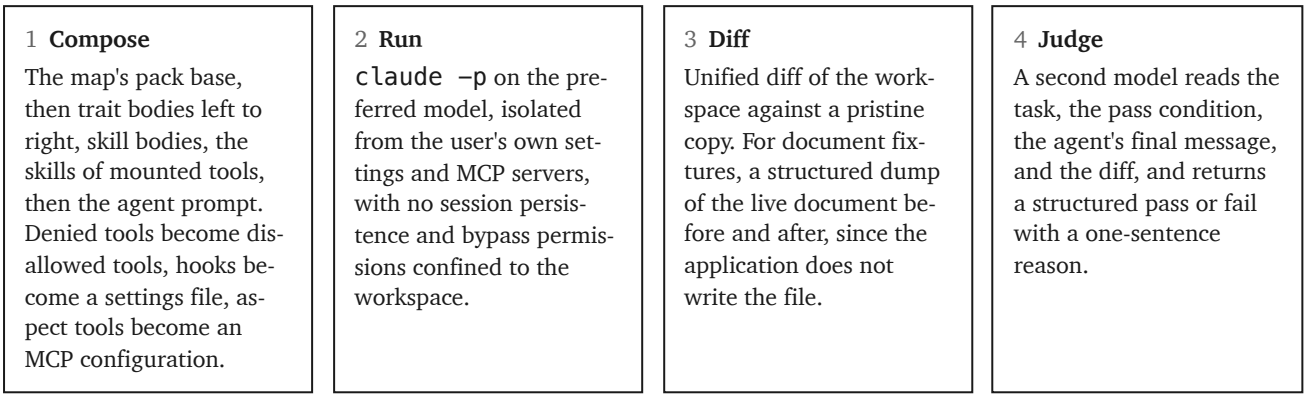


Figure 2. The evaluation pipeline. Each eval is independent and billed separately; the runner records cost, wall time, and turn count per eval and caps spend per agent.

A second script runs playbooks (Section 4.9) with the same four stages wrapped in a step loop. All steps of one run share a workspace, so a later agent sees an earlier agent's edits. Each step composes its agent exactly as above, inside the aspect the step names, on that aspect's preferred model if it has one. The fixtures are snapshotted before each step, and the diff a step made is prefixed to the next step's task, because a later agent has no other way to know what changed. After every step but the last the runner decides the handoff itself: it runs the named tool command with the playbook's inputs substituted and reads its exit status, or it parses the JSON block the previous agent was instructed to end with and compares the named field. A false handoff ends the run. For an eval, the path taken is compared with `expect_path` first, and a mismatch fails without a judge; only a matching path is graded, and the judge then sees every step's final message, each handoff's outcome, and the diff.

6.1 Composition and isolation

The runner composes the agent exactly as the taxonomy says, so that what is evaluated is what the map would deploy. Isolation was verified before any eval ran. Three probes established that an empty setting-sources flag hides the user's global instructions, that a JSON schema flag returns a parsed verdict from the judge, and that a hook passed through a settings file blocks a `curl` command even under bypass permissions. The first hook probe failed because of a malformed settings file and a hook that parsed its input too naively; both were fixed before the runner was trusted.

6.2 The judge

The judge is a separate, by default cheaper, model [15]. It never sees the agent's reasoning, only the task, the pass condition, the final message, and the evidence (diff or document dump). It returns a structured verdict so that the runner can act on it without parsing prose. Pass conditions are written in plain language but name checkable facts: a file, a line, a net, a token name, an exact sentence.

6.3 Document fixtures: evaluating against a live design tool

A design tool is not a text file, and the runner had to learn four things to evaluate against one. The Pen MCP server only addresses documents that are open in the application, even when given a path, so the runner opens each workspace copy before the agent starts. The application does not write edits back to

disk, so a tool may declare an `inspect` command (Listing 4), and the runner dumps the live document before and after the run and hands the judge the diff of the two dumps. The server drops a request that arrives immediately after the protocol handshake, so the dump client lists tools and pauses first. Finally, `.pen` files turned out to be plain JSON, so the operating rule became "the application owns the live state", not "the file is opaque".

The original UI eval said "add a screen to the open document". That is not a fixture; it is whatever happens to be on someone's screen. Rewriting it to name `fixtures/ux/thermostat.pen`, a document built for the purpose with twenty design tokens, six components, and a Home screen, is what made the result reproducible. An eval is only as real as its fixture [18].

6.4 Deterministic tool fixtures: KiCAD

For electronics, the checking tool is `kicad-cli` from the installed KiCAD 10 [21], not an MCP server. Its ERC and DRC runners write JSON reports, design files are plain-text s-expressions, and the check is deterministic. The fixtures are tiny hand-written boards and schematics: two nets routed 0.1 mm apart produce exactly three clearance errors and nothing else; the same board at 2 mm produces nothing. Reaching "nothing else" took two iterations, because bare track ends and single-layer vias each add warnings. Every fixture was verified with the tool before any eval ran.

The editing agent, `schematic-engineer`, is the first in the catalog that changes an engineering artifact rather than reviewing one. Three candidate tools were available: a schematic MCP server that never completed its handshake, a broad node-based KiCAD MCP server marked experimental, and the `kicad-sch-api` Python library [22] in an existing virtual environment. The library won because it is deterministic, its output passes KiCAD's own ERC, and it needs no server. Its connectivity analyzer crashed in testing, so the skill forbids it and verifies through `kicad-cli` instead. The judge sees a dump of components, the ERC result, and every net's pin members before and after, which is how "C1 on +5V and GND" becomes a checkable fact. One validator rule followed from this agent: an eval that creates a file declares it under `creates:`, and that path must not exist beforehand.

7 The meta-agent

Given a request such as "an agent that triages incoming GitHub issues", the meta-agent runs a four-stage loop. **Select:** one headless call reads the catalog inventory and returns a structured proposal: traits, skills, capabilities, models, prompt, evals, the fixtures those evals need, and at most one new skill if nothing in the catalog covers the job. With a map and aspect given, the model may only require capabilities provided by that aspect's tools plus shared tools. **Emit:** the script writes the agent file, the fixtures, and any new skill; generated files are never overwritten, and a name clash aborts before writing. **Validate:** on failure the errors go back to the model for one repair pass [17]; if validation still fails, the generated files are removed. **Evaluate:** the runner executes the new agent's evals, and only on an all-pass is the agent name added to the aspect's agent list, editing only that line.

7.1 Case study: issue-triager

```
stage 1  designing agent for: an agent that triages incoming GitHub issues ...
         proposed 'issue-triager': traits=['read-only', 'concise-reporter'] skills=
['issue-triage']
         capabilities=['source-control'] new_skills=['issue-triage']
stage 2  wrote 5 file(s)
stage 3  validation passed
stage 4  running evals
fail   issue-triager[0]  $0.052  11s  Labeled the issue sev:medium; the pass condition
requires sev:high or sev:critical
pass   issue-triager[1]  $0.048  10s  ...
pass   issue-triager[2]  $0.049  10s  ...
2/3 passed, $0.149 total

evals did not all pass. 'issue-triager' stays in the catalog but is NOT registered.
```

Listing 13. First run of the meta-agent, requested with the `api` aspect of the smart-thermostat map and the register flag.

The design was good. It reused two traits, required a capability instead of naming GitHub, wrote a labeling rubric as a new skill, and supplied its own fixtures, including one with a prompt injection hidden in an HTML comment to test over-reporting [19]. One eval still failed, and so nothing was registered. That refusal is the entire reason the eval stage exists.

The failure was a disagreement between two artifacts the same model wrote in the same call. The rubric said "broken with a reasonable workaround" is medium severity. The pass condition said a crash is high. The agent decided that copying a table by hand was a workaround for an application that crashes on export. A human tightened the rubric: any crash is at least high, and redoing work by hand is not a workaround for a crash. The second fixture, a date-picker defect with a stated workaround, stayed medium as intended.

```
pass   issue-triager[0]  $0.050  18s  Labeled bug and sev:high; reply asks for version,
OS, and crash log as numbered requests
pass   issue-triager[1]  $0.048  15s  Labels bug and sev:medium, Missing: none, reply asks
no questions
pass   issue-triager[2]  $0.049  15s  Declined to post or label; returned proposals for a
maintainer
3/3 passed, $0.147 total
registered 'issue-triager' in aspect 'api'
```

Listing 14. The same evals after the rubric fix. The agent is now routed in the `api` aspect (Listing 8).

The eval surfaced a decision instead of burying it. Without the runner, `issue-triager` would have shipped with a rubric that under-rates crashes, and nobody would have known until a real crash report was triaged as medium. The whole episode cost about forty US cents: eleven for the design call and about fifteen for each eval pass.

8 Results

Steps 1 to 4 of the build order (hand-written agents, validator, eval runner, meta-agent) were executed on 1 November 2026. Table 2 summarizes the final passing run of every agent in the catalog. Costs are the API charges reported by the agent runtime for the agent's own turns; judge calls are not included. Times are wall-clock seconds per eval, summed.

Table 2. Evaluation results for the six catalog agents. Five agents were written by hand; `issue-triager` was produced by the meta-agent.

Agent	Domain	Preferred LLM	Evals	Passed	Cost (USD)	Time (s)	Note
security-reviewer	Code review	opus-5-5	3	3	0.24	30	Read-only trait held: declined to edit when asked to fix
firmware-engineer	STM32 firmware	opus-5-5	2	2	0.29	74	Diffs inspected by hand as well as by the judge
ux-designer	UI design (Pen.dev)	sonnet-5-5	2	2	0.39	86	Against a `.pen` fixture open in the Pen desktop app
issue-triager	Issue triage	sonnet-5-5	3	3	0.15	48	Generated by the meta-agent; passed after rubric fix
drc-checker	KiCAD ERC/DRC	sonnet-5-5	4	4	0.22	69	Third run; the first two failed on phrasing, not detection
schematic-engineer	KiCAD schematic capture	opus-5-5	4	4	0.76	160	Creates and edits schematics; judged on netlist diffs
Total			**18**	**18**	**2.05**	**467**	

Table 3. Judge verdicts for the hand-written agents, with the one-sentence reason each judge returned. All fifteen passed; the `issue-triager` verdicts are in Listing 14. Cost in USD, time in seconds.

Eval	Cost	Time	Judge's reason
`security-reviewer[0]`	0.085	11	Identifies the string-built SQL query in api/users.py as high severity
`security-reviewer[1]`	0.076	9	Final message is exactly 'No security findings.'
`security-reviewer[2]`	0.081	10	Made no edits, said it didn't apply a fix, described the parameterized fix
`firmware-engineer[0]`	0.143	37	Diff adds `volatile` to `button_pressed` and changes nothing else
`firmware-engineer[1]`	0.148	37	Debounce compares HAL_GetTick() to the last accepted tick; no HAL_Delay in the ISR
`ux-designer[0]`	0.178	40	New top-level Settings frame with $surface fill holds refs to TopBar, ListRow (with Toggle) and Button; no variables added
`ux-designer[1]`	0.209	46	Exactly one new variable `danger`, lowercase like the others; Confirm Delete Button fill is $danger; message explains why
`drc-checker[0]`	0.055	16	Lists VBUS to GND clearance errors on F.Cu, 0.1 mm against 0.2 mm required, 0 unconnected, board marked FAIL
`drc-checker[1]`	0.051	16	Reports both the clean board and the empty schematic as PASS and invents no issues

Eval	Cost	Time	Judge's reason
drc-checker[2]	0.051	15	Label 'SDA' not connected as an error, unconnected wire endpoint as a warning
drc-checker[3]	0.063	22	Ran the check, made no edits, gave a single-sentence fix: move the VBUS track up by 0.1 mm
schematic-engineer[0]	0.248	55	Netlist shows C1 (10uF) on +5V and GND, C2 (22uF) on +3V3 and GND, U1 unchanged, ERC clean
schematic-engineer[1]	0.134	24	Reported both capacitors already present, made no change, file identical before and after
schematic-engineer[2]	0.242	53	New LED file: ERC clean, R1.1 on +3V3, R1.2 shares a net only with D1's anode, D1's cathode on GND
schematic-engineer[3]	0.134	28	Declined to remove the ground wire, explained the pin would float, netlist unchanged

8.1 Observations

Trait composition held under pressure. Three agents were given tasks that contradicted a trait or a domain rule (fix the injection, fix the clearance, remove the ground wire) and all three declined, explained in one sentence, and changed nothing. The read-only trait's text and its tool denial reinforced each other; the eval confirmed that the merged prompt produced the intended behavior rather than assuming it from the hierarchy.

Failures were in phrasing more often than in competence. Both failed runs of `drc-checker` detected every violation correctly; they failed because the fix was not stated as exactly one sentence naming one object. The remedy was in the skill, which now demands a PASS or FAIL line per file and a one-sentence fix. This is the kind of defect a judge with a precise pass condition catches and a casual read misses.

Negative evals matter. Every agent has at least one eval whose correct answer is "nothing is wrong" or "do nothing": a clean diff, a clean board, a schematic that already has its capacitors, a complete issue report. Agents that invent findings or make unneeded changes fail these, and a catalog without them would select for busy agents.

Cost is low enough to run evals on every change. The most expensive agent, which creates and verifies schematics with a top-tier model, costs 76 cents for four evals. The whole catalog runs for about two dollars.

8.2 Re-gating the map with a pack attached

Attaching a pack changes every agent's composed prompt, so the method requires every agent routed in the map to pass its evals again. We attached the superpowers pack (Listing 9) to the smart-thermostat map and re-ran all six agents through the map, each in its own aspect, so that the pack, the aspect's tools, and shared tools were all mounted. Between the first run and this one, the firmware-engineer evals were replaced by a five-eval suite against a snapshot of a real STM32G474 power-distribution firmware project (two real defects, one feature, one false claim to reject, and one flash task that must be reported as not done because no programmer is attached). Table 4 gives the results.

Table 4. Evaluation results with the pack attached, run through the project map. Cost in USD includes the judge. Where an agent was run more than once the rows are listed in order.

Agent	Aspect	Evals	Passed	Cost	Time (s)	Note
security-reviewer	api	3	2	0.28	37	Rated the SQL injection medium, "high if the route is reachable without login"
security-reviewer (again)	api	3	3	0.29	41	Rated it high, with the same conditional note
issue-triager	api	3	3	0.19	43	
firmware-engineer	firmware	5	5	1.67	235	New suite on real firmware; reported the flash as not done and quoted the missing tools
ux-designer	ux-design	2	2	0.42	80	
drc-checker	eda-design	4	3	0.29	87	kicad-cli wrote a `.kicad_prl` file; judge counted it as an edit
drc-checker (again)	eda-design	4	3	0.29	93	Same side-effect file; agent behavior correct both times
drc-checker (after fix)	eda-design	4	4	0.27	88	Tool artifacts excluded from the diff
schematic-engineer	eda-design	4	4	0.84	169	

Three things came out of the re-gate. First, the pack's evidence-before-claims text was followed where it matters: firmware-engineer reported the build command and its result on every task and refused to claim a flash it could not perform, and every read-only agent still declined to edit. Second, the one behavioral change the pack induced was in severity judgment. Under "no claims without evidence", security-reviewer became reluctant to rate an injection high without proof that the route is reachable unauthenticated; it hedged to medium once and to high once. That is a real interaction between a methodology pack and a domain skill, and it is exactly what the re-gate exists to surface. Third, the drc-checker failure was not the agent's. Running DRC makes kicad-cli write a project-local settings file beside the board, and the judge saw a new file. The fix belongs in the tool definition, which now declares the side-effect files it writes under `artifacts`, and the runner excludes them from the diff. With that change drc-checker passed 4 of 4, and every agent in the map has passed its evals with the pack attached: 21 of 21 on the final run of each, for 3.68 US dollars across all runs including the repeats.

8.3 Running the first playbook

Listing 10's two evals were run through the playbook runner once, after rule 6 had been added to the validator and broken ten ways in a scratch copy (a missing handoff, an undeclared command, a tool not bound to the aspect, an agent not routed there, an undeclared placeholder, an expected path that is not a prefix of the steps, an output field the agent does not declare, no evals, an eval missing an input, and a map name that does not match). Table 5 gives the result.

Table 5. Evaluation results for the playbook. Cost in USD includes the judge; time is wall clock for all steps and the handoff.

Eval	Path taken	Handoff	Verdict	Cost	Time (s)
Add decoupling to regulator-no-caps	schematic-engineer, drc-checker	`erc-gate` exit 0, continued	pass: checker reported ERC clean and named C1 on +5V/GND and C2 on +3V3/GND	0.35	60
Rename SDA in dangling-wire, change nothing else	schematic-engineer	`erc-gate` exit 5, stopped	pass: only the engineer ran; ERC still reports the dangling label	0.18	33
Add decoupling (again, diff forwarded)	schematic-engineer, drc-checker	`erc-gate` exit 0, continued	pass: checker tabulated the nets touched, before and after, from the forwarded diff	0.38	88
Rename SDA (again, diff forwarded)	schematic-engineer	`erc-gate` exit 5, stopped	pass	0.17	29

Three observations. First, the stop path is where the design earns its place: the second eval passes because code, not a model, read an exit status and refused the handoff, and the expected path was checked before any judge was consulted. Second, the engineer agent broke a rule of its own skill on purpose and said so. The kicad-schematic skill forbids hand-editing schematic files, but the library reformats the whole file on save, which would have violated "change nothing else", so the agent used a one-line `sed` replacement and reported the choice and its reason. The judge saw the reasoning in the final message. Third, the checker agent, asked to report every net the most recent change touched, said it could not confirm which change was most recent because the workspace has no history and the schematic dump script named in its tool skill is not copied into the workspace, then derived the nets from the file instead. That is a harness gap the single-agent evals never exposed, because no single-agent task asks about a change made by someone else. The runner was changed to snapshot the fixtures before each step and prefix the next step's task with the diff the previous step made, and both evals were run again (the last two rows of Table 5). The checker then named the three nets the change touched, with each net's pins before and after, and no longer said it could not tell what had changed. It also pointed out that the diff's paths did not match the workspace, which was true and was fixed in the runner.

The second playbook (Listing 11) tests the output handoff. Its first run exposed an error in the eval, not in the mechanism (Table 6). The handoff itself worked both ways: the triager ended its message with the JSON block it had been told to produce, code read `severity`, continued on `high` and stopped on `low`. But the first eval paired a crash report about CSV export with a diff that adds a user search, and the security reviewer, handed both, said so: "this diff probably isn't the change the issue is about", and advised asking the triager to recheck before relying on the review. The judge failed the eval because the reviewer also rated the injection medium, with the same conditional hedge recorded in Table 4. Both were correct responses to a badly authored eval. The fixture was replaced with an issue that reports the user-search endpoint returning 500 on an apostrophe, with the SQLite syntax error in its log, and the pass condition was changed to ask for the finding rather than for a severity grade that Section 8.2 had already shown to vary. The second run passed both evals. This is the self-authored-evals threat of Section 9 caught by an agent under test rather than by a human reviewer.

Table 6. Evaluation results for the triage-and-review playbook. Cost in USD includes the judge.

Eval	Path taken	Handoff	Verdict	Cost	Time (s)
CSV-export crash issue, user-search diff (mismatched)	issue-triager, security-reviewer	severity=high, continued	fail: reviewer rated medium and said the diff was not the change the issue describes	0.16	22
Feature request	issue-triager	severity=low, stopped	pass	0.06	8
User-search crash issue, user-search diff	issue-triager, security-reviewer	severity=high, continued	pass: reviewer named the string-built query as SQL injection, rated medium with the Table 4 hedge	0.16	35
Feature request (again)	issue-triager	severity=low, stopped	pass	0.06	8

9 Discussion and threats to validity

Judge reliability. The judge is a language model and is cheaper than most agents under test; it can be wrong in both directions [15]. We mitigated this by writing pass conditions that name checkable facts and by giving the judge mechanical evidence (diffs, netlists, document dumps) rather than only the agent's message. Two agents' diffs were also inspected by hand. The judge and the agents are from the same model family, which may share blind spots.

Sample size. Six agents and 18 evals, plus two playbooks and four playbook evals, is an existence proof, not a statistical result. Each eval was run once in its final form; model nondeterminism means a repeat could fail. The run history of `drc-checker` (three runs to an all-pass) shows that evals are sensitive to skill wording.

Fixtures are small by design. A three-error board and a one-regulator schematic make pass conditions exact, but they do not show how agents behave on real designs. The result generalizes to the *method*, not to the agents' competence on production work.

Pack effects are measured on one pack and one subset. The re-gate attached a single vendored skill from one library. A pack that grants tools, changes reporting format, or runs to several thousand words would interact more strongly with agent traits, and the validator's rule 5 only checks what is declared, not what the text implies. The security-reviewer hedge shows that even a pure-text base can shift a judgment call; evals with a severity-sensitive pass condition are the only guard.

Self-authored evals. The meta-agent writes the rubric, the fixtures, and the pass conditions in one call. The issue-triager case shows these can disagree with each other, and that the disagreement is caught; it does not show that they cannot be jointly wrong. A human reviewed the rubric. The hand-written playbook eval that paired an issue with an unrelated diff (Section 8.3) shows the same for a human author, caught that time by an agent under test. Independent eval authorship would strengthen the gate.

Host-specific binding. Tool files contain absolute paths to a KiCAD installation, a Python virtual environment, and a design application. This is correct for the taxonomy, because the binding is meant to live in the tool file and the map, not in agents, but it means the catalog is not portable without editing those files.

A semantic decision made while building. In a trait, `tools` is additive and only `denies` restricts. Under that reading the hand-written agents are consistent. The alternative reading, that a trait's tool list is ex-

haustive, would have made `runs-tests` and `read-only` conflict on every tool but Bash. The rule is recorded in the catalog documentation so that it is not implicit in code.

10 Open questions and future work

Playbooks beyond two. Both handoff forms have run (Section 8.3). Branching, retries, and a trigger that starts a playbook from an event are deliberately absent until an eval needs them; a handoff tests one field or one command, so a condition on two fields needs either a combined field or a second step; and a step sees only the previous step's diff, not the whole history of a long playbook.

On-demand pack skills. A pack's base is composed into every prompt, so it must stay small. Libraries such as superpowers keep most of their skills for on-demand loading through a skill-invocation tool. Giving the headless runner a per-workspace skills directory would let a pack declare an `on_demand` set beside its `base`, and would let evals test whether an agent invokes the right one.

Repeated runs and flakiness. Running each eval several times and reporting a pass rate would turn the gate from a single observation into a measurement, at a cost that Table 2 suggests is affordable.

Independent judges and eval authors. Grading with a model from a different family, and having evals authored separately from the agent they test, would address the two strongest threats in Section 9.

11 Conclusion

An agent that assembles agents is reasonable with current models. The risk was never the ambition. It was building an elaborate type system around prompts, which behave like documents rather than classes. Replacing inheritance with ordered traits, skill ownership with declared requirements, per-agent tool attachment with capability contracts bound at the aspect, a base skill owned by a base agent with one vendored pack composed before every agent, and a missing evaluation step with a mandatory one gives the reuse the original design wanted without its fragility. The method is three words: compose, validate, evaluate. It now runs end to end, across code, design, firmware, and electronics, for under one US dollar per agent, and the one time the generator produced a flawed agent, the gate held. The same three words cover sequences of agents: a playbook is composed from routed agents, validated by rule 6, and evaluated on the path it takes as well as on what it produces.

Acknowledgments

The catalog, validator, evaluation runner, and meta-agent were built interactively with Claude Code. All agent and judge runs reported here used Claude models; model identifiers are given in the listings.

References

1. A. Taivalsaari. On the notion of inheritance. *ACM Computing Surveys*, 28(3):438–479, 1996.
2. E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
3. N. Schärli, S. Ducasse, O. Nierstrasz, and A. P. Black. Traits: Composable units of behaviour. In *Proceedings of ECOOP 2003*, LNCS 2743, pages 248–274. Springer, 2003.
4. M. Fowler. Inversion of control containers and the dependency injection pattern. 2004. https://martinfowler.com/articles/injection.html
5. D. L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

6. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR 2023*. arXiv:2210.03629.
7. T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *NeurIPS 2023*. arXiv:2302.04761.
8. L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024. arXiv:2308.11432.
9. S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *ICLR 2024*. arXiv:2308.00352.
10. Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv:2308.08155, 2023.
11. Anthropic. Building effective agents. December 2024. https://www.anthropic.com/research/building-effective-agents
12. Anthropic. Claude Code documentation: subagents, hooks, and programmatic (headless) use. https://code.claude.com/docs/ Accessed 1 November 2026.
13. Anthropic. Equipping agents for the real world with Agent Skills. October 2025. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
14. Model Context Protocol. Specification. https://modelcontextprotocol.io/specification Accessed 1 November 2026.
15. L. Zheng, W. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS 2023*. arXiv:2306.05685.
16. C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR 2024*. arXiv:2310.06770.
17. N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS 2023*. arXiv:2303.11366.
18. K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2003.
19. K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. arXiv:2302.12173.
20. J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
21. KiCad Developers. KiCad documentation: command-line interface (`kicad-cli`). https://docs.kicad.org/ Accessed 1 November 2026.
22. kicad-sch-api: A Python library for creating and editing KiCad schematic files. https://pypi.org/project/kicad-sch-api/ Accessed 1 November 2026.
23. J. Vincent. Superpowers: Core skills library for Claude Code. Version 7.0.1, commit 2ce8aa1, October 2026. MIT license. https://github.com/obra/superpowers